CHƯƠNG 4:
ĐA HÌNH (POLYMORPHISM)

ĐA HÌNH

- Là 1 trong những đặc điểm chính của phương pháp lập trình hướng đối tượng.
- Tính đa hình cho phép một phương thức có **các tác động khác nhau trên nhiều loại đối tượng khác nhau**.
- Đối tượng sẽ **thể hiện đúng hành vi tính chất của đối tượng mà nó trỏ (gán) đến**

ĐA HÌNH

Cụ thể khi ta gán 1 đối tượng x của lớp cha bằng đối tượng y của lớp con thì đối tượng x phải thể hiện hành vi tính chất của đối tượng y

- Đối tượng x thể hiện hành vi tính chất của x.
- Gán x=y thì x thể hiện hành vi tính chất của y
- Gán x=z thì x thể hiện hành vi tính chất của z

VÍ DỤ

- Khi khai báo đối tượng Người x là Sinh viên thì x thể hiện hành vi tính chất của 1 sinh viên.
- Khi khai báo đối tượng Người x là nhân viên thì x thể hiện hành vi tính chất của 1 nhân viên.
- Khi khai báo động vật x là 1 con mèo thì x kêu “meo meo”.
- Khi khai báo động vật x là 1 con chó thì x kêu “gâu gâu”.

VÍ DỤ

- Lớp động vật có phương thức kêu (xuất ra dòng chữ “aaa”)
- Lớp mèo kế thừa từ lớp động vật, cũng có phương thức kêu (xuất ra dòng chữ “meo meo”)
- Khi ta khai báo động vật là một con mèo thì nó phải kêu meo meo
- dv1 là instance của động vật
- m1 là instance của mèo
- dv1=m1
- Khi gọi dv1.keu() kết quả như thế nào?

CÁCH CÀI ĐẶT

- Các phương thức của lớp con có cùng tên, tham số và kiểu trả về với phương thức của lớp cha → **Overriding** (ghi đè/phủ quyết)
- Để cho rõ ràng, thêm chú thích @Override vào trước khai báo phương thức trùng tên của lớp con (không bắt buộc)
- @Override: che hàm đã được định nghĩa lớp cha và định nghĩa lại hàm cho lớp con

```
public class DONGVAT
{
	String ten;
	float cannang;
	public DONGVAT()
	{
		ten="DV";
		cannang=0;
	}
	public DONGVAT( String t, float cn)
	{
		this.ten=t;
		this.cannang=cn;
	}
	public void keu()
	{
		System.out.print("\n AAA!!!");
	}
	public void xuat()
	{
		System.out.print("\nTen: "+ten+"\nCan nang: "+cannang+" Kg");
	}
}
```

```
class MEO extends DONGVAT
{
	String maulong;
	public MEO() {
		super();
		maulong = "";
	}
	public MEO (String ten, float cannang, String maulong)
	{
		super(ten, cannang);
		this.maulong = maulong;
	}
	@Override public void keu()
	{
		System.out.print("\n MEO MEO!!!");
	}
	@Override public void xuat()
	{
		super.xuat();
		System.out.print("\n mau long: “+ maulong);
	}
}
```

```
class CHO extends DONGVAT
{
	String maulong;
	public CHO() {
		super();
		maulong = "";
	}
	public CHO (String ten, float cannang, String maulong)
	{
		super(ten, cannang);
		this.maulong = maulong;
	}
	@Override public void keu()
	{
		System.out.print("\n GAU GAU!!!");
	}
	@Override public void xuat()
	{
		super.xuat();
		System.out.print("\n mau long: “+ maulong);
	}
}
```

```
public static void main(String[] args)
{
    DONGVAT[] dv = new DONGVAT[3];//mang 3 dong vat
    dv[0] = new DONGVAT();
    dv[0].keu();
    dv[1] = new CHO("Lulu",15,"vang");
    dv[1].keu();
    dv[2] = new MEO("Doremon",3,"xanh");
    dv[2].keu();
}
```

KẾT QUẢ XUẤT RA MÀN HÌNH LÀ GÌ?

TOÁN TỬ INSTANCEOF

- Toán tử instanceof: kiểm tra kiểu của đối tượng (trả về true hoặc false)

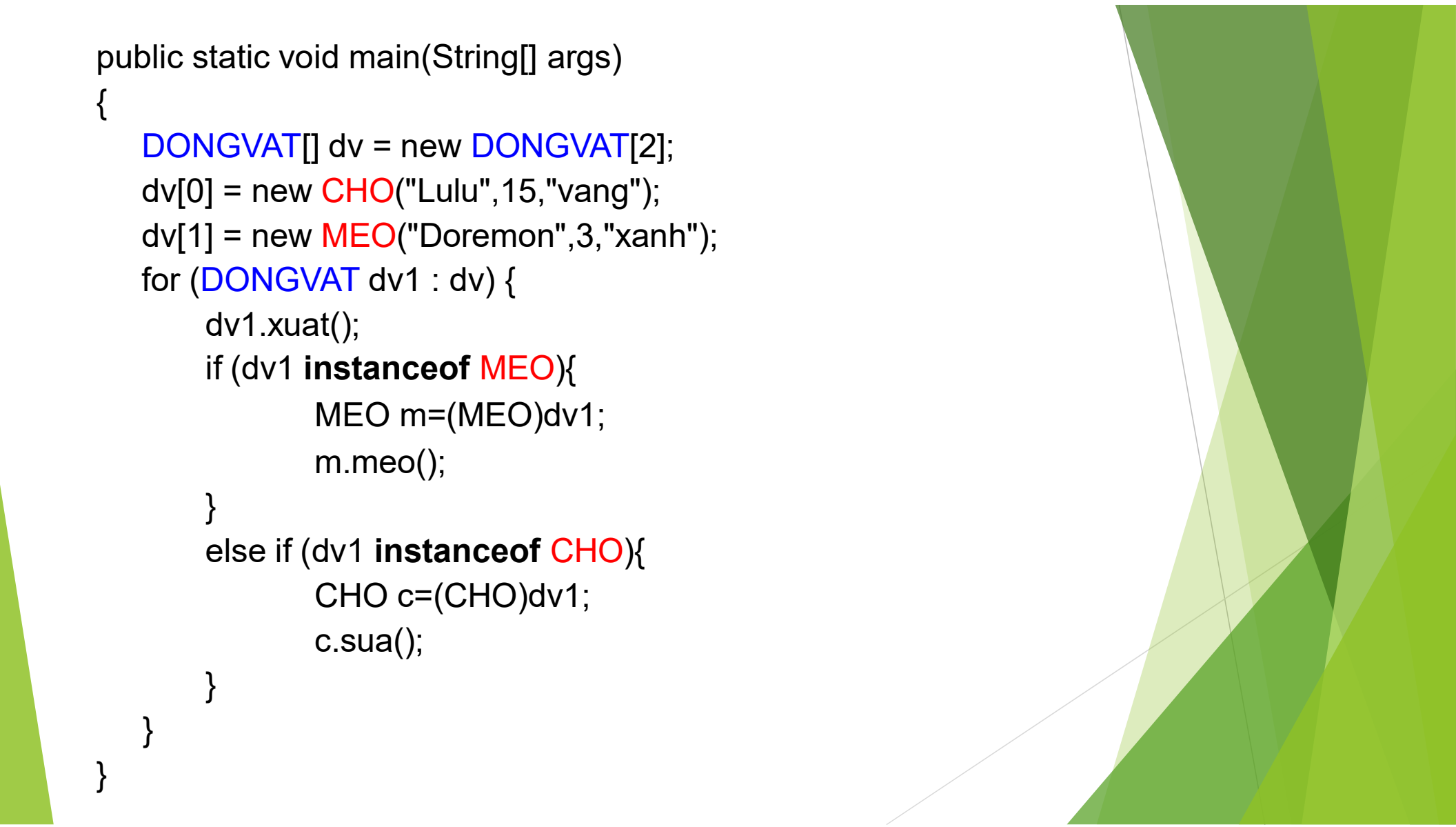
Ví dụ:

```
if (dv instanceof MEO) {
        // đối tượng kiểu mèo
}
if (dv instanceof CHO) {
        // đối tượng kiểu chó
}
```

```
class MEO extends DONGVAT
{
	String maulong;
	public MEO() {
		super();
		maulong = "";
	}
	public MEO (String ten, float cannang, String maulong)
	{
		super(ten, cannang);
		this.maulong = maulong;
	}
	public void meo()
	{
		System.out.print("\n MEO MEO!!!");
	}
	@Override public void xuat()
	{
		super.xuat();
		System.out.print("\n mau long: “+ maulong);
	}
}
```

```
class CHO extends DONGVAT
{
	String maulong;
	public CHO() {
		super();
		maulong = "";
	}
	public CHO (String ten, float cannang, String maulong)
	{
		super(ten, cannang);
		this.maulong = maulong;
	}
	public void sua()
	{
		System.out.print("\n GAU GAU!!!");
	}
	@Override public void xuat()
	{
		super.xuat();
		System.out.print("\n mau long: “+ maulong);
	}
}
```

```
public static void main(String[] args)
{
    DONGVAT[] dv = new DONGVAT[2];
    dv[0] = new CHO("Lulu",15,"vang");
    dv[1] = new MEO("Doremon",3,"xanh");
    for (DONGVAT dv1 : dv) {
        dv1.xuat();
        if (dv1 instanceof MEO){
            MEO m=(MEO)dv1;
            m.meo();
        }
        else if (dv1 instanceof CHO){
            CHO c=(CHO)dv1;
            c.sua();
        }
    }
}
```



LỚP FINAL

- Là lớp không cho kế thừa.
- Ví dụ:

```
final class A {
…
}
```

CÁC LỚP LỒNG NHAU

- Là lớp nằm trong 1 lớp khác nhằm hỗ trợ cục bộ lớp chứa nó.
- Các lớp được chứa bên trong được gọi là lớp lồng (nested class).
- Lớp chứa được gọi là lớp ngoài (outer class).
- Lớp lồng bên trong có khả năng truy cập đến tất cả các thành viên của lớp ngoài.
- Lớp lồng bên trong có thể ẩn đối với tất cả các lớp khác (private), nếu lớp lồng là lớp public thì có thể được truy xuất thông qua lớp ngoài theo cú pháp <lớp outer>.<lớp nested>

VÍ DỤ

```
class A {
        class B{
                int value;
                .....
}
```

```
class A {
        public class B{
                private int value;
                public B() {
                        .....
                }
        }
}
```

HÀM TRỪU TƯỢNG (ABSTRACT METHOD)

- Hàm trừu tượng là hàm không có sự thực thi.

Khai báo:

[thuộc tính truy xuất] abstract [kieu] <tenham>();

- Ví dụ:

Xây dựng hàm keu() của lớp động vật không thực thi:

public abstract void keu();

LỚP TRỪU TƯỢNG (ABSTRACT CLASS)

- Lớp trừu tượng là lớp không có đối tượng cụ thể, nó dùng để cho các lớp khác kế thừa nó.
- Lớp trừu tượng chứa ít nhất một hàm trừu tượng.
- Các hàm trừu tượng này sẽ được thực thi ở lớp dẫn xuất.
- Không thể tạo thể hiện cho lớp trừu tượng.

LỚP TRỪU TƯỢNG (ABSTRACT CLASS)

- Khai báo:

  [thuộc tính truy xuất] abstract class <tên lớp>

- Ví dụ

  + Xây dựng lớp trừu tượng động vật, lớp mèo kế thừa từ lớp động vật

  public abstract class DONGVAT { …. }

  class MEO extends DONGVAT { …. }

```
public abstract class DONGVAT
{
        String ten;
        float   cannang;
        public DONGVAT()
        {
                ten="DV";
                cannang=0;
        }
        public DONGVAT(String t,float cn)
        {
                this.ten=t;
                this.cannang=cn;
        }
        public abstract void keu();
}
```

```
class MEO extends DONGVAT
{
	String maulong;
	public MEO()
	{
			super();
			maulong = "";
	}
	public MEO (String ten, float cannang, String maulong)
	{
			super(ten,cannang);
			this.maulong = maulong;
	}
	public void keu()
	{
			System.out.print("\n MEO MEO!!!");
	}
}
```

```
public static void main(String[] args)
{
    DONGVAT[] dv = new DONGVAT[2];
    dv[1] = new MEO("Doremon",3,"xanh");
    dv[1].keu();
}
```

LUYỆN TẬP

- Xây dựng lớp hình học là lớp trừu tượng có hàm trừu tượng: tinhdientich(); tinhchuvi(); nhap(); xuat();
- Xây dựng lớp hình tròn, lớp hình chữ nhật kế thừa từ lớp hình học và hiện thực hóa các hàm trừu tượng trong lớp hình học.
- Trong hàm main, xây dựng mảng gồm các đối tượng lớp hình học và test các phương thức.